

Database Indexing - Interview Ready Guide

1. What is an Index?

An **index** is a database structure that improves the speed of data retrieval operations on a table at the cost of additional storage and write overhead. Think of it like the index of a book — it helps you find data quickly without reading the whole table.

Internal working: - Most databases use **B-Trees** (or variations like B+ Trees) for indexing. - B-Trees maintain sorted data and allow **logarithmic time complexity** ($O(\log n)$) for search, insert, and delete. - Without an index, scanning a whole table is $O(n)$, but an index allows direct access to rows. - Queries use the index to find **row pointers** and fetch data efficiently.

2. How Index Works Internally

1. Database reads all values of the indexed column.
2. Sorts them in a **tree-like structure**.
3. Stores **pointers to the corresponding rows**.

Example:

Table: Users
Columns: id, name, email
Index: email_idx on email

Searching WHERE email='abc@example.com' uses the index to directly locate the row instead of scanning all rows.

Internal Index Types: - **B-Tree:** Balanced tree, good for equality and range queries. - **Hash:** Fast for exact matches, poor for range queries. - **Bitmap:** Efficient for low-cardinality columns. - **Full-Text:** Optimized for text search.

3. Types of Indexes and Definitions

1. **Primary Index**
 - o Automatically on **primary key**.
 - o Ensures **uniqueness**.
 - o Usually a **clustered B-Tree**.
2. **Unique Index**
 - o Ensures column(s) have **unique values**.
 - o Can be applied to non-primary columns.

3. **Composite (Multi-Column) Index**

- o Index on **multiple columns**.
- o Optimized for queries filtering by all indexed columns.

4. **Clustered Index**

- o Table rows physically sorted to match the index.
- o Only one per table.
- o Fast range queries.

5. **Non-Clustered Index**

- o Separate from table data.
- o Stores **row pointers** to table.
- o Multiple allowed per table.

6. **Full-Text Index**

- o Optimized for **text search**.
- o Supports keyword or phrase searching with relevance ranking.

7. **Bitmap Index**

- o Uses **bit arrays** for keys.
- o Efficient for **low-cardinality columns**.
- o Common in data warehouses.

8. **Hash Index**

- o Uses a **hash table**.
 - o Fast for **exact-match queries**, not good for ranges.
-

4. Advantages of Indexing

- Faster **SELECT queries**.
- Efficient **JOIN operations**.
- Enforces **uniqueness** constraints.

Trade-offs: - Extra **disk space** usage. - Slower **INSERT/UPDATE/DELETE** due to index maintenance.

5. How to Explain in an Interview

“An index in a database is like an index in a book — it allows quick lookup of rows without scanning the entire table. Internally, databases use B-Trees where data is stored in a balanced tree for $O(\log n)$ search. Indexes store pointers to actual rows. Types include primary, unique, composite, clustered, non-clustered, bitmap, full-text, and hash, each optimized for different queries. Trade-offs are extra storage and slower writes, but read performance improves drastically.”